

# **COMP2300/6300 Practice Mid-Semester Exam Questions**

Semester 1 2017

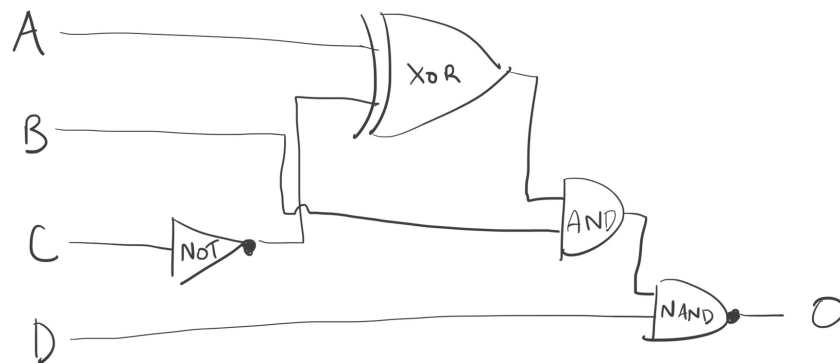
This is not a practice exam—there's more stuff here than you could hope to finish in a 90min exam. It's just a list of questions to test your understanding.

Questions are not equally weighted, and the sizes of answer boxes do not necessarily relate to the number of marks given for the question.

Remember that you can bring a cheat sheet (one double-sided a4 page).

Greater marks will be awarded for simple, concrete answers than vague, rambling ones. Marks may be deducted for providing information that is irrelevant to a question.

---

**Question 1:**

Draw the full truth table for this gate configuration:

Answer:

## Question 2:

```
.syntax unified
.global main
```

**main:**

```
    mov r0, #1
```

**label1:**

```
    lsls r0, #1
    bcs label2
    b label1
```

**label2:**

```
    rrxs r0, r0
    beq main
    b label2
```

### Part 1

In one English sentence, what does this ARM assembly program do?

Answer:

### Part 2

If this program is allowed to run, will the value in **r0** ever stabilise, or will it keep changing? Explain your answer.

Answer:

### Question 3:

Oh no—you accidentally spilled lemonade on your discoboard and the **overflow bit** (in the program status register **xPSR**) has stopped working, although everything else is still working.

Use the **qadd** and **add** instructions together to determine whether **r0+r1** will overflow?

@ your code starts here (the values to add are already in r0 and r1)

#### overflow:

@ your code should branch here if there is overflow

**b overflow**

#### no\_overflow:

@ your code should branch here if there's no overflow

**b no\_overflow**

## Question 4:

You've just landed a job as an instruction set designer for a new CPU architecture. The new design has 4 CPU registers and needs to have 8-bit opcodes.

### Part 1

Design an 8-bit opcode for your CPU's **add** instruction. Include a diagram of the meaning of each opcode bit, and a syntax example (i.e. like you're writing a cheat sheet for your new CPU architecture).

*Answer:*

**Part 2**

Your boss tells you that new CPU will often be required to perform 3-argument addition in a single instruction (e.g.  $Rd := Rm + Rn + Ro$ ). Is this possible? Explain your answer.

*Answer:*

**Question 5:**

You might have noticed that the **cbz** instruction can't jump backwards. Write a **my\_cbz** macro with the same syntax as the **cbz** instruction but can jump to any label.

*Answer:*

**Question 6:**

Write an ARM assembly program which returns the absolute value of a signed integer argument, i.e.  $f(x) = |x|$ .

*Answer:*



## Question 7:

This program contains a **remainder13** recursive function which takes a single argument (passed in **r0**) and returns (again, in **r0**) the remainder of that argument when divided by 13.

```
.syntax unified
.global main

remainder13:
    stmdb sp!, {r0, lr}
    mov r1, r0

    cmp r1, #13
    blt end_remainder13

    sub r0, r1, #13

    bl remainder13
    mov r1, r0

end_remainder13:
    ldmbia sp!, {r0, lr}

    mov r0, r1
    bx lr

main:
    mov r0, #61
    bl remainder13

end:
    nop
    b end
```

**Part 1**

In the program on the previous page, how many times will the `remainder13` function be called? Draw the full stack state for each call to `remainder13`.

*Answer:*

**Part 2**

Write a generalised **remainder** version of this function which takes the remainder “base” (e.g. 13 in the previous version) as a second argument.

*Answer:*